

SmartHub Bid API — Reference

The SmartHub serving API exposes the bid-recommendation model over HTTP. It has two functional endpoints plus a health check:

Method	Path	Purpose
POST	/recommend_bid	Return the expected-profit-maximizing bid for one lead.
POST	/explain_bid	Explain an already-logged prediction (SHAP + optional LLM narrative).
GET	/health	Liveness + which model is loaded.

- **Base URL (staging):** `http://<host>:8000` (served via nginx → uvicorn).
- **Content type:** `application/json` for all `POST` requests.
- **Auth:** none at the edge today (network-restricted).
- **Every prediction is logged** server-side to `smarthub_prediction_log` with a unique `prediction_id`, which is returned in the response. SHAP is attached to that row asynchronously (see `/explain_bid`). See `docs/PREDICTION_LOG_SCHEMA.md` for the log schema.

1. POST /recommend_bid

Runs one lead through the bidding policy (`decide_bid`) and returns the recommended bid. The response is **lean by default**; pass `verbose: true` to also receive the full decision payload used by the logging schema.

1.1 Request body (`BidRequest`)

Required parameters — the request is rejected (HTTP 422) if any is missing. A lead that cannot supply all of these is treated as an upstream data-quality issue for SmartHub to fix, not a case the model should attempt to score.

Field	Type	Constraints	Description
<code>expected_revenue</code>	number	<code>> 0</code>	Reject-discounted expected revenue if the lead is won. Drives the profit calc.
<code>campaign_id</code>	integer		Campaign the lead belongs to.
<code>lead_type_id</code>	integer	<code>6</code> or <code>1</code>	Which lead-type model to use (<code>6</code> =auto, <code>1</code> =home). No default — must be supplied.
<code>account_id</code>	integer		Partner/account the lead belongs to.
<code>source_type_id</code>	integer		Lead source / sub-source id (the warehouse source-step / <code>sol_step</code> id).
<code>lead_ping_id</code>	integer		Correlation key back to <code>public.lead_pings.id</code> . Required in the payload, but never used in the decision — only for logging / joining realized outcomes.
<code>created_at</code>	datetime	<code>YYYY-MM-DD HH:MM:SS</code> (Pacific)	Date and time the lead was created. The model's timing features (<code>created_hour</code> , <code>created_day</code> , <code>is_workday</code>) are derived from this .

Required lead attributes — all lead types (present on both auto and home; the request is rejected if missing)

Field	Type	Description
<code>insured</code>	string	Whether the lead is currently insured.
<code>age</code>	number	Applicant age.

Required lead attributes — **auto** (`lead_type_id` = 6) (auto-specific; required only for auto requests)

Field	Type	Description
<code>home_owner</code>	string	Whether the (auto) lead is also a homeowner — a buyer bundling filter (auto model feature, not the home lead type).
<code>num_vehicles</code>	number	Number of vehicles (drives "multiple vehicles").
<code>num_auto_accidents</code>	number	Number of auto accidents.
<code>dui</code>	string	DUI history.
<code>sr22_required</code>	string	Whether an SR-22 filing is required.

(Home leads (`lead_type_id` = 1) have their own required attribute set — TBD once the home product tiers are shared.)

Optimizer controls (optional; sensible defaults)

Field	Type	Default	Constraints	Description
<code>target_cm</code>	number	<code>0.25</code>	$0 \leq x < 1$	Target contribution margin (sets the bid ceiling).
<code>min_bid</code>	number	<code>0.25</code>	≥ 0	Partner-side minimum / lowest candidate bid.
<code>bid_step</code>	number	<code>0.25</code>	> 0	Increment between candidate bids in the sweep.

Optional context

Field	Type	Default	Description
<code>verbose</code>	boolean	<code>false</code>	When <code>true</code> , return the full decision payload (\$1.3). Not a feature — never affects the bid.

Optional lead features (default `null` ; missing values are imputed by the model pipeline)

Field	Type	Field	Type
<code>state</code>	string	<code>num_drivers</code>	number
<code>gender</code>	string	<code>num_auto_violations</code>	number
<code>marital_status</code>	string	<code>continuous_coverage_months</code>	number
<code>military_affiliation</code>	string		
<code>traffic_tier</code>	string		

Required vs optional (updated 2026-08-20). The required set is the core contract fields (above) plus the buyer-facing lead attributes that gate pricing/eligibility. Attributes present on **both** auto and home (`insured` , `age`) are required on every request; the auto-specific attributes (`home_owner` , `num_vehicles` , `num_auto_accidents` , `dui` , `sr22_required`) are required only when `lead_type_id` = 6 . Rationale: a lead missing any of these represents an upstream data-quality issue SmartHub should resolve, not a case the model should score — validation shows these are not missing on non-erred rows, and such rows typically lack `expected_revenue` anyway. Any field not listed above is ignored.

Why context is passed explicitly rather than looked up from `lead_ping_id`. The decision needs campaign / account / source / lead-type / timing at request time; resolving them from `lead_ping_id` would require a warehouse

lookup that adds ~1 s to the turnaround, which the ≤ 1 s SLA can't absorb. `lead_ping_id` is therefore required for logging only.

These required-field and lookup assumptions are pending confirmation with Kiran (DS sync, Tue).

Example request

```
curl -s -X POST http://localhost:8000/recommend_bid \
-H 'Content-Type: application/json' \
-d '{
  "expected_revenue": 100,
  "campaign_id": 40088,
  "lead_type_id": 6,
  "account_id": 221,
  "source_type_id": 574,
  "lead_ping_id": 998877,
  "created_at": "2026-08-14 14:23:05",
  "home_owner": "true",
  "insured": "true",
  "age": 34,
  "num_vehicles": 2,
  "num_auto_accidents": 0,
  "dui": "false",
  "sr22_required": "false",
  "traffic_tier": "tier_2",
  "state": "CA"
}'
```

1.2 Response — default (lean)

Returned when `verbose` is omitted or `false`. Optimized for the hot path.

```
{
  "recommended_bid": 10.25,
  "recommended_bid_predicted_win_rate": 0.9715,
  "recommended_bid_predicted_profit": 87.19,
  "max_bid": 75.0,
  "n_candidate_bids": 300,
  "decision_path": "model",
  "decision_reason": "Standard profit-maximizing bid from the currently-serving model.",
  "model_data_age_days": 0,
  "prediction_id": "d0e044ab-2ba8-4454-88ad-19331f4560a9",
  "lead_ping_id": null
}
```

Field	Type	Description
<code>recommended_bid</code>	number null	The chosen bid. <code>null</code> = no viable bid (revenue too low to bid at/above the floor while meeting <code>target_cm</code>).
<code>recommended_bid_predicted_win_rate</code>	number null	Model-predicted win probability at the recommended bid.
<code>recommended_bid_predicted_profit</code>	number null	Predicted profit at the recommended bid ($\text{win_rate} \times (\text{expected_revenue} - \text{bid})$).
<code>max_bid</code>	number	CM-respecting bid ceiling used for the sweep.
<code>n_candidate_bids</code>	integer	Number of candidate bids evaluated by the optimizer.
<code>decision_path</code>	string	<code>model</code> <code>cold_start_fallback</code> <code>exploration</code> (see §1.4).
<code>decision_reason</code>	string	Plain-English explanation of the path taken.
<code>model_data_age_days</code>	integer null	Age (days) of the serving model's training data; flags staleness (informational).
<code>prediction_id</code>	string (uuid4)	Receipt for this prediction's log row; use it with <code>/explain_bid</code> .
<code>lead_ping_id</code>	integer null	Echoed back exactly as sent.

1.3 Response — verbose (`verbose: true`)

Includes **all lean fields above**, plus the full logging-schema payload below. The data is packaged from values already computed during the prediction (no extra model inference), so it doesn't change the decision — only the response size. The candidate sweep is **capped to 20** on the wire (the selected bid + the first 19 by bid); the full sweep is still persisted to the log. **SHAP is not included** — retrieve it later via `/explain_bid` using `prediction_id`.

Additional fields:

Field	Type	Description
<code>served_at</code>	string (ISO-8601)	When the prediction was served (UTC).
<code>tat_seconds</code>	number	Server turnaround time in seconds.
<code>package_version</code>	string	SmartHub package version (semver) that served the row.
<code>lead_type_id</code> / <code>lead_type_name</code>	integer / string	Which lead-type model answered (e.g. <code>6</code> / <code>"auto"</code>).
<code>campaign_id</code> / <code>account_id</code> / <code>source_type_id</code>	integer null	Business context echoed back.
<code>input_features</code>	object	Raw request snapshot (the lead features as received).
<code>model_input_features</code>	object	The exact row the model scored (post feature-engineering, imputed).
<code>feature_cols</code>	array[string]	Ordered feature columns the serving model used.
<code>expected_revenue</code> / <code>target_cm</code> / <code>min_bid</code> / <code>bid_step</code>	number	Optimizer inputs echoed back.
<code>candidate_bid_generation</code>	object	How the candidate set was built: <code>{method, min_bid, max_bid, bid_step, n_candidates}</code> .
<code>model_name</code>	string	Human-friendly production slug (e.g. <code>auto_v8</code>).
<code>model_version</code>	string	Exact training-run id (e.g. <code>run_20260817T105906Z_2b0f9075</code>).
<code>model_uri</code>	string	Path/URI of the loaded artifact.
<code>model_type</code>	string	e.g. <code>lightgbm</code> .
<code>model_calibrated</code>	boolean	Whether probability calibration is applied.
<code>training_table_version</code>	string	Training-table snapshot the model came from.
<code>model_data_min_created_at</code> / <code>model_data_max_created_at</code>	string	Date range of the training data.
<code>recommended_bid_predicted_cm</code>	number null	Predicted contribution margin at the recommended bid (<code>profit</code> / <code>expected_revenue</code>).
<code>serving_config</code>	object	Serving-policy snapshot: <code>{exploration_variance_pct, recency_window_days, cold_start_fallback_bid_pct}</code> .
<code>candidate_evaluations</code>	array[object]	Capped to ≤20. Each entry: <code>{bid, predicted_win_rate, expected_profit, selected}</code> ; exactly one has <code>selected: true</code> (the chosen bid).

Example (verbose, abridged)

```
{
  "recommended_bid": 10.25,
  "recommended_bid_predicted_win_rate": 0.9715,
  "recommended_bid_predicted_profit": 87.19,
  "recommended_bid_predicted_cm": 0.8719,
  "decision_path": "model",
  "prediction_id": "7d12c4aa-9ede-4c29-8fef-cde14190c614",
  "served_at": "2026-08-17T11:13:03.601597+00:00",
  "package_version": "0.1.0",
  "model_name": "auto_v8",
  "model_version": "run_20260817T105906Z_2b0f9075",
  "model_type": "lightgbm",
  "model_calibrated": true,
  "training_table_version": "2026-08-17T083011Z",
  "feature_cols": ["bid", "age", "...", "current_carrier"],
  "input_features": { "campaign_id": 13, "state": "CA", "age": 34 },
  "model_input_features": { "bid": 0.25, "age": 34.0, "state": "CA" },
  "candidate_bid_generation": {
    "method": "equally_spaced", "min_bid": 0.25, "max_bid": 75.0,
    "bid_step": 0.25, "n_candidates": 300
  },
  "serving_config": {
    "exploration_variance_pct": 0.1, "recency_window_days": 30,
    "cold_start_fallback_bid_pct": 0.5
  },
  "candidate_evaluations": [
    {"bid": 0.25, "predicted_win_rate": 0.0876, "expected_profit": 8.7393, "selected": false},
    "... first 19 by bid ...",
    {"bid": 10.25, "predicted_win_rate": 0.9715, "expected_profit": 87.1932, "selected": true}
  ]
}
```

1.4 Decision paths

decision_path	Meaning
model	Normal case — profit-maximizing bid from the serving model.
cold_start_fallback	No model promoted yet for this lead type; bids a configured fraction of the way from <code>min_bid</code> to the ceiling. <code>recommended_bid_predicted_*</code> are <code>null</code> .
exploration	Scheduled probe — the model's optimal bid is perturbed $\pm$ <code>exploration_variance_pct</code> on a reproducible hour-of-week schedule.

2. POST /explain_bid

Explains a **previously logged** prediction by id. It loads the exact model version that served that prediction and computes SHAP on the exact scored row — it **never re-runs the bid decision**, so the explanation always matches the logged bid. The explanation is also persisted back onto the same log row.

2.1 Request body (`ExplainRequest`)

Field	Type	Required	Default	Description
<code>prediction_id</code>	string	☐	—	The <code>prediction_id</code> returned by <code>/recommend_bid</code> .
<code>with_llm</code>	boolean		<code>true</code>	Include a natural-language <code>explanation</code> (LLM narrative).
<code>with_bid_curve</code>	boolean		<code>true</code>	Include the predicted win-rate/profit curve around the bid.

Example request

```
curl -s -X POST http://localhost:8000/explain_bid \
-H 'Content-Type: application/json' \
-d '{"prediction_id": "7d12c4aa-9ede-4c29-8fef-cde14190c614", "with_llm": false}'
```

2.2 Response

```
{
  "base_prediction": 0.114,
  "prediction": 0.9715,
  "feature_contributions": [
    {"feature": "num_auto_accidents", "value": 1, "contribution": 0.84},
    {"feature": "age", "value": 34.0, "contribution": -0.31}
  ],
  "top_factors": [
    {"feature": "num_auto_accidents", "value": 1, "shap": 0.84, "direction": "increased"},
    {"feature": "age", "value": 34.0, "shap": -0.31, "direction": "decreased"}
  ],
  "base_win_rate": 0.114,
  "bid_curve": [
    {"bid": 8.0, "predicted_win_rate": 0.93, "expected_profit": 85.5}
  ],
  "explanation": "The recommended bid is driven up mainly by ...",
  "prediction_id": "7d12c4aa-9ede-4c29-8fef-cde14190c614",
  "lead_ping_id": null
}
```

Field	Type	Description
<code>base_prediction</code>	number null	Model's average win rate before this lead's factors (log-odds base, expressed as a probability).
<code>prediction</code>	number null	The served (calibrated) win rate for this lead — equals the logged <code>recommended_bid_predicted_win_rate</code> .
<code>feature_contributions</code>	array[object]	Every feature's SHAP contribution: <code>{feature, value, contribution}</code> , sorted by <code> contribution </code> desc. Contributions are in log-odds units.
<code>top_factors</code>	array[object]	Top-N view: <code>{feature, value, shap, direction}</code> (<code>direction</code> = <code>increased</code> / <code>decreased</code>).
<code>base_win_rate</code>	number null	Same as <code>base_prediction</code> ; kept for existing consumers.
<code>bid_curve</code>	array[object]	Only if <code>with_bid_curve=true</code> — predicted metrics across nearby bids.
<code>explanation</code>	string	Only if <code>with_llm=true</code> — natural-language narrative generated from the SHAP payload.
<code>prediction_id</code>	string	Echoed back.
<code>lead_ping_id</code>	integer null	From the logged row.

Note on numbers: `feature_contributions` + `base_prediction` sum to the model's *uncalibrated* output, so they won't exactly reconstruct the *calibrated* `prediction`. Use contributions for *why* (direction/magnitude) and `prediction` for the served win probability.

Robustness: the explanation never 500s — if SHAP can't run (e.g. an unsupported model type or an unloadable artifact), it returns empty `feature_contributions` / `top_factors` with an `explanation` describing why.

3. GET /health

Liveness probe and current model status.

```
curl -s http://localhost:8000/health
```

```
{
  "status": "ok",
  "lead_type_id": 6,
  "model_uri": "/app/data/models/auto/run_20260817T105906Z_2b0f9075.pkl",
  "model_loaded": true
}
```

Field	Type	Description
<code>status</code>	string	<code>"ok"</code> when the app is up.
<code>lead_type_id</code>	integer	Default lead type checked.
<code>model_uri</code>	string null	Resolved artifact path for the default model.
<code>model_loaded</code>	boolean	Whether that model is currently cached in memory. <code>false</code> right after startup until the first request lazy-loads it.

4. Errors

Status	When	Body
422 Unprocessable Entity	Request fails validation (missing required field, malformed <code>created_at</code> , <code>expected_revenue ≤ 0</code> , wrong type).	FastAPI validation detail listing the offending fields.
404 Not Found	<code>/explain_bid</code> with an unknown <code>prediction_id</code> .	<code>{"detail": "No prediction logged for id '...'."}</code>
500 Internal Server Error	Unexpected serving failure (e.g. model artifact fails to load). The failure is logged with a <code>status: "error"</code> prediction-log row.	<code>Internal Server Error</code> .

5. Notes & conventions

- **`prediction_id` is a receipt.** `/recommend_bid` generates and returns it before the log row is written (the write is decoupled off the request path), so a lookup by it can rarely be empty if the async insert failed — treat it as "the id we intend to log under."
- **Logging never affects serving.** Prediction-log writes and SHAP enrichment happen off the response path; a logging outage never delays or fails a bid.
- **`verbose` adds no inference** and doesn't change latency on the default (lean) path — only opt-in callers pay the (small) serialization cost.
- **SHAP is asynchronous.** For `/recommend_bid` , SHAP is backfilled onto the log row by a separate worker; to read it on demand, call `/explain_bid` with the `prediction_id` .